Discrete Event System Project: RideHailing

DANIAL MOAFI

October 2025

1 Introduction

This project presents a simplified simulation of a ride-hailing platform modeled as a discrete event system (DES). In this system, two main entities interact dynamically over time: **drivers** and **passengers**. Each event in the simulation represents a key system action such as a passenger request, driver acceptance, ride completion, or driver becoming available again.

The motivation behind this work is to analyze how variations in the number of active drivers influence key system metrics such as the acceptance rate, total number of completed rides, and overall driver profit. By simulating these interactions within a controlled DES framework, it becomes possible to study the system's behavior under different load conditions and identify strategies that can improve operational efficiency and fairness between drivers and passengers.

2 System Design

2.1 City Zoning and Demand Distribution

For starting the simulation, the city is divided into different zones based on the passenger request arrival rate. To make the state design and analysis simpler, the model currently uses three zones. However, this setup is flexible and can be expanded to include more zones in future simulations.

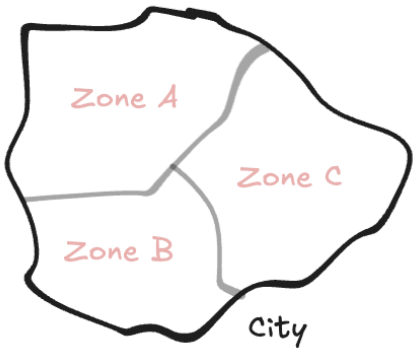

Figure 1: Division of the city into three zones based on demand levels.

2.2 Demand Model (Poisson Arrivals)

Passenger requests are modeled as a Poisson process per origin–destination (OD) pair. Let zones be $\{A, B, C\}$ and let $\lambda_{i\to j}$ denote the arrival rate of requests from zone i to zone j.

Inter-arrival times for $i \to j$ are exponential:

$$T_{i\to j} \sim \text{Exp}(\lambda_{i\to j}), \quad \mathbb{E}[T_{i\to j}] = \frac{1}{\lambda_{i\to j}}.$$

Rates can be estimated from data by counting requests over an observation window $[0, T]$:

$$\hat{\lambda}_{i\to j} = \frac{\#\text{requests}_{i\to j}}{T}.$$

For simplicity, rates are assumed stationary.

2.3 Travel Time Model

The ride time represents the travel duration for a driver to move from one zone to another (e.g., from A to B). It reflects the traffic pattern between zones and directly affects driver availability after completing a ride.

In this simulation, ride times are modeled as an exponential random variable with a fixed rate parameter $\mu_{i\to j}$ for each origin–destination pair:

$$T^{\text{ride}}_{i\to j} \sim \text{Exp}(\mu_{i\to j}), \quad \mathbb{E}[T^{\text{ride}}_{i\to j}] = \frac{1}{\mu_{i\to j}}.$$

This assumption simplifies the system by capturing the variability of travel times while maintaining analytical tractability.

2.4 Driver Model

For simplicity, all drivers are assumed to have identical behavior. Each driver alternates between working and resting periods, both modeled as exponential random variables:

$$T^{\text{work}} \sim \text{Exp}(\lambda_{\text{work}}), \quad T^{\text{rest}} \sim \text{Exp}(\lambda_{\text{rest}}).$$

During their working hours, drivers may accept or reject incoming ride offers. The acceptance rate is fixed and identical across all drivers, representing the probability of accepting a new request when available.

3 Simulation

3.1 Events

A total of 11 event types are defined, covering passenger requests, ride completions, and driver availability changes.

- **Request A→B**, **Request A→C**, **Request B→A**, **Request B→C**, **Request C→A**, **Request C→B**:
  Passenger requests from one zone to another, generated as Poisson processes with rates $\lambda_{i\to j}$.
- **Ride Finish AB**, **Ride Finish AC**, **Ride Finish BC**:
  A ride is completed, and the driver becomes available in the destination zone. Ride duration follows an exponential distribution with rate $\mu_{i\to j}$.
- **Online**: A driver becomes available for accepting new requests.
- **Offline**: A driver stops working and becomes unavailable.

For simplicity, the pickup time between driver and passenger is neglected, and driver actions (acceptance, going online/offline) are considered instantaneous. Also, if no driver is available in the requested origin zone at the time of a request, the request is immediately rejected.

3.2 States

The system state is represented using two matrices that describe the status of all drivers in the network.

- **Online/Ride Matrix (M_{on}):** Each row and column correspond to the city zones. The diagonal elements indicate the number of available drivers in each zone, while the off-diagonal cells represent ongoing rides from an origin to a destination zone.
- **Offline Matrix (M_{off}):** The diagonal elements represent the number of offline drivers resting in each zone. All off-diagonal elements are zero, since offline drivers are not engaged in any rides. When a driver's resting time ends, their count is transferred back to the corresponding diagonal cell of M_{on}.

The number of possible system states depends on the number of drivers in the simulation. To keep the model simple, the simulation considers only one driver. The total number of possible states can be computed combinatorially using the *Stars and Bars* method, which determines how drivers can be distributed among the defined states.

For a single driver, the total number of distinct states is 12. For three drivers, this number increases rapidly to 165, illustrating how the state space grows exponentially with the number of drivers.

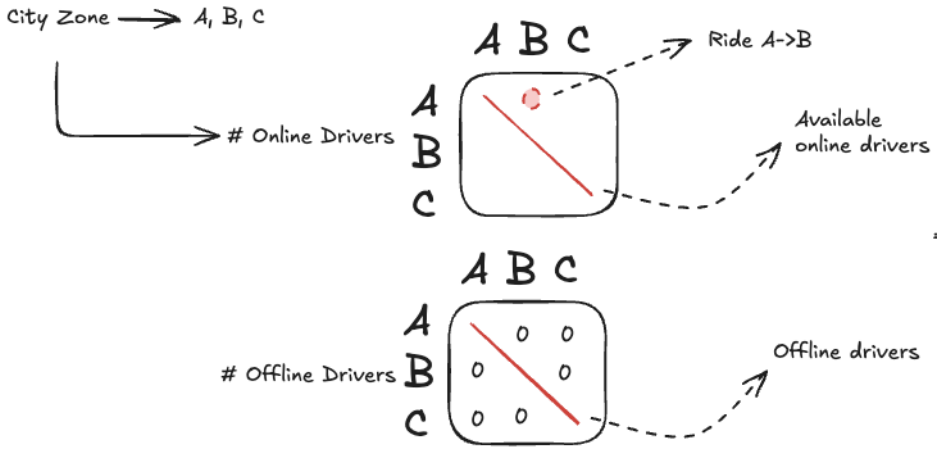

Figure 2: Matrix representation of system states

These 12 configurations cover all possible conditions for a single driver in the simulation.

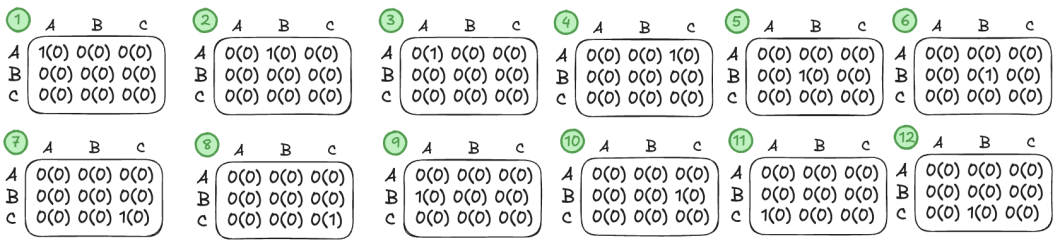

Figure 3: All possible system states for a single driver

3.3 State Transition Diagram

Based on the defined events and possible states, the system can be represented as a state transition diagram.

The simulation starts with the driver available in zone A as the initial state.

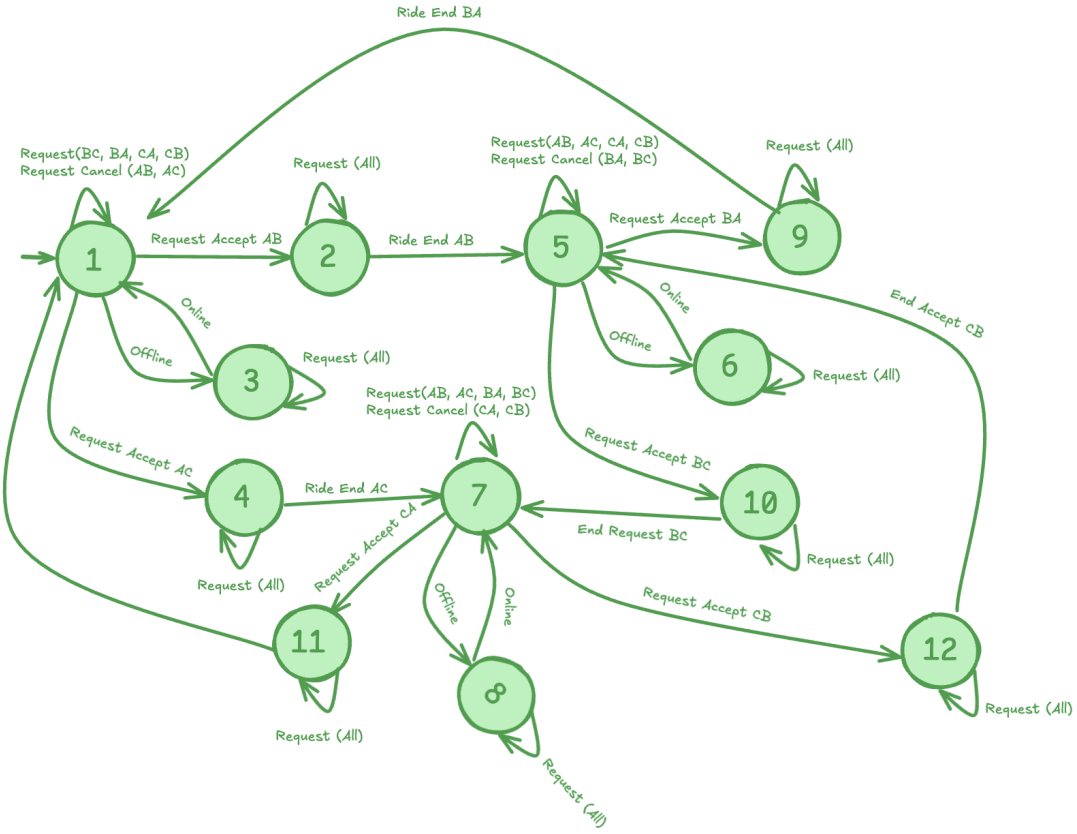

Figure 4: State transition diagram

4 Simulation

4.1 Parameters

The following parameters were used in the simulation. All rates are expressed in events per second, and time values represent expected means derived from the corresponding exponential distributions.

The simulation starts with one driver available in zone A, while zones B and C initially have no drivers.

4.2 Steady-State Analysis

The event-driven model with exponential clocks defines a finite CTMC.

The steady-state (limit) probabilities $\pi \in \mathbb{R}^{12}$ solve

$$\pi Q = 0, \tag{1}$$

$$\sum_{s=1}^{12} \pi_s = 1, \qquad \pi_s \geq 0. \tag{2}$$

Table 1: Simulation parameters for the ride-hailing discrete event system.

Parameter	Description	Value / Range
$\lambda_{A\to B}$	Request rate from A to B	0.00278 (mean 6 min)
$\lambda_{A\to C}$	Request rate from A to C	0.00167 (mean 10 min)
$\lambda_{B\to A}$	Request rate from B to A	0.00238 (mean 7 min)
$\lambda_{B\to C}$	Request rate from B to C	0.00208 (mean 8 min)
$\lambda_{C\to A}$	Request rate from C to A	0.00333 (mean 5 min)
$\lambda_{C\to B}$	Request rate from C to B	0.00167 (mean 10 min)
$\mu_{A\to B}$	Service rate (ride A→B)	1/450 (mean 7.5 min)
$\mu_{A\to C}$	Service rate (ride A→C)	1/900 (mean 15 min)
$\mu_{B\to A}$	Service rate (ride B→A)	1/450 (mean 7.5 min)
$\mu_{B\to C}$	Service rate (ride B→C)	1/675 (mean 11.25 min)
$\mu_{C\to A}$	Service rate (ride C→A)	1/900 (mean 15 min)
$\mu_{C\to B}$	Service rate (ride C→B)	1/675 (mean 11.25 min)
λ_{work}	Work period rate (mean 1.5 h)	1/5400
λ_{rest}	Rest period rate (mean 25 min)	1/1500
p_{accept}	Request acceptance probability	0.7
T_{sim}	Simulation duration	8 hours

We compute π by solving the linear system in (1) with the normalization (2).

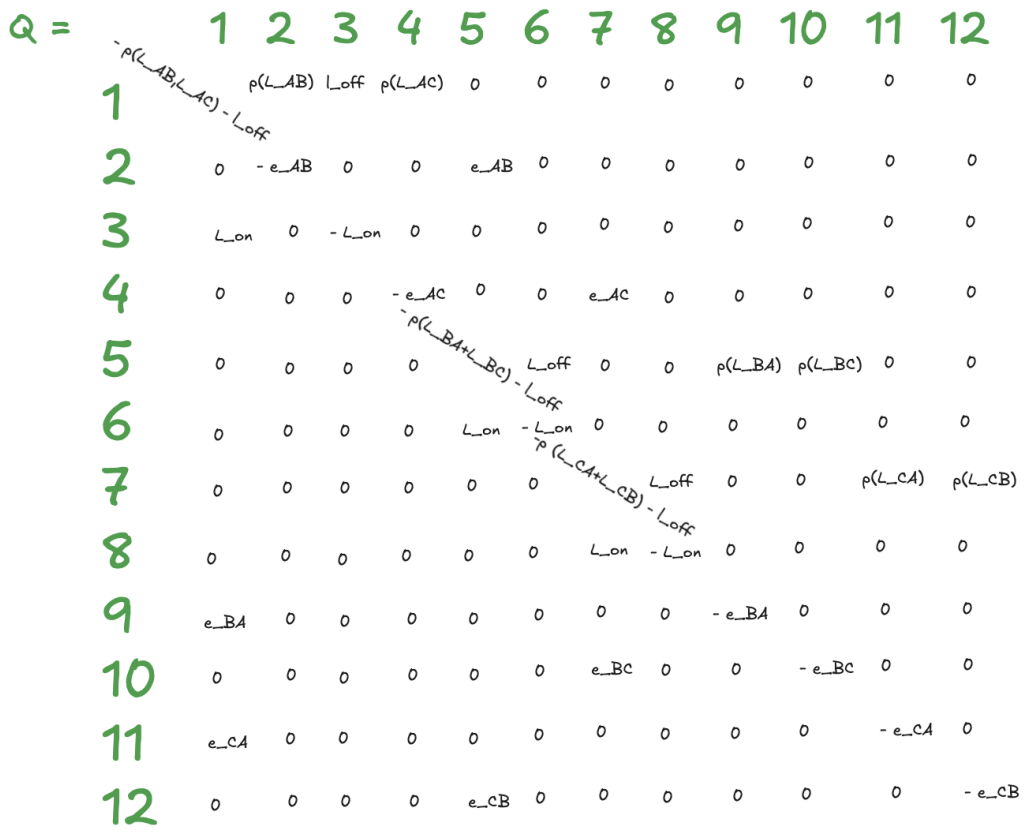

Figure 5: Transition rate matrix Q

The comparison shows an excellent match between analytical and simulated results. All

Table 2: Comparison between analytical steady-state probabilities and simulation estimates after 5000 hours.

State ID	Analytical π_i	Simulated $\hat{\pi}_i$	Abs. Error $\lvert\pi_i - \hat{\pi}_i\rvert$
1	0.0526	0.0518	0.0008
2	0.0663	0.0678	0.0015
3	0.0748	0.0723	0.0025
4	0.0701	0.0701	0.0000
5	0.0553	0.0555	0.0002
6	0.1471	0.1472	0.0001
7	0.0869	0.0885	0.0016
8	0.0884	0.0893	0.0009
9	0.0663	0.0662	0.0001
10	0.1050	0.1026	0.0024
11	0.0874	0.0889	0.0015
12	0.0998	0.0999	0.0001

absolute errors are below 3×10^{-3}, confirming that the event-driven simulation converges to the theoretical steady-state distribution. The small differences are due to stochastic variability in the simulation and finite observation time.

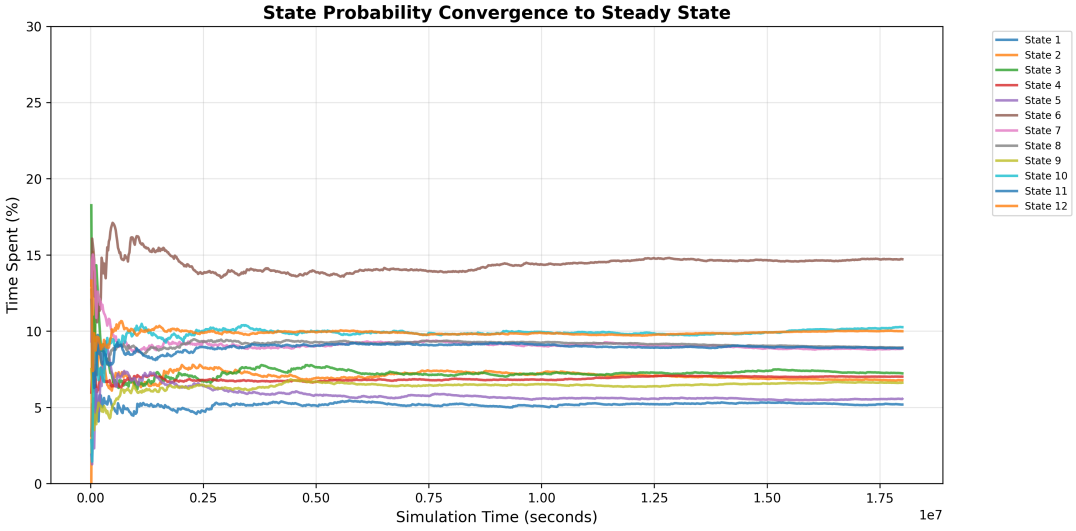

Figure 6: Transition rate matrix Q

4.3 Impact of the Number of Drivers on System Performance

To evaluate the impact of the number of available drivers on the system performance, the simulation was run for a fixed time horizon of 10 hours while gradually increasing the number of drivers from one to eight. As expected, the overall acceptance rate rises steadily with the number of drivers, since more requests can be served simultaneously and the probability of having an available driver in each zone increases. However, the average number of completed rides per driver decreases, indicating that higher driver supply leads to idle time and lower utilization per driver. The detailed statistics are summarized in Table 3.

Table 3: Simulation statistics for different numbers of drivers over a 10-hour horizon.

Drivers	States	Requests	Accepted	Rejected	Completed	Acceptance (%)	Events	Accepted / Driver
1	12	161	27	134	26	16.8	205	27.0
2	77	190	71	119	70	37.4	283	35.5
3	363	199	86	113	86	43.2	339	28.7
4	1364	238	121	117	119	50.8	433	30.3
5	4367	230	127	103	126	55.2	460	25.4
6	12375	238	156	82	153	65.5	512	26.0
7	31823	240	169	71	164	70.4	559	24.1
8	75581	258	187	71	184	72.5	599	23.4

5 Conclusions

This project modeled a simplified ride-hailing system as a discrete-event simulation, implemented in **Python**. Analytical steady-state probabilities from the CTMC model were validated through long-run simulations, showing strong agreement. Experiments with different numbers of drivers demonstrated that the acceptance rate increases with supply, while rides per driver decrease due to lower utilization. A version with realistic (non-exponential) ride times confirmed that the system admits a steady state. The complete implementation and data analysis scripts are available at:

`https://github.com/Danialmoa/DISystem_RideHailing`